

Module - 3 (Analysis and System Design)

Analysis: Object Modeling - Identifying Object Classes - Preparing a Data Dictionary- Identifying Associations. Dynamic Modeling - Preparing a Scenario - Interface Format – Identifying Event- Building a State Diagram. Functional Modeling - Identifying input and Output Values – Building DataFlow Diagram - Describing Functions. System Design: Breaking System into Subsystems, Identifying Concurrency, Allocating Subsystems to Processors and Tasks, Managing Data Stores, Handling of Global Resources, Common Architectural Framework.

Analysis Phase

- Write a problem statement.
- Build an Object Model:
 - Object Model = object model diagram + data dictionary
- Develop a Dynamic Model:
 - Dynamic Model = state diagrams + global event flow diagram
- Construct a Functional Model:
 - Functional Model = data flow diagrams + constraints
- Verify, iterate, and refine the three models:
 - Analysis Document = Problem statement + Object Model + Dynamic Model + Functional Model.

Problem statement

- The first step to develop anything is **to state the requirements**.
- The problem statement should state **what is to be done and not how it is to be done**.
- It should be a statement of needs, not a proposal for a solution.
- The requestor should indicate which features are mandatory and which are optional; to avoid overly constraining design decisions.
- The requestor should avoid describing system internals.

- A problem statement may have more or less detail.
- Most problem statements are ambiguous, incomplete or even inconsistent.
- Some requirements are wrong.
- Some requirements have unpleasant consequences on the system behavior or impose unreasonable implementation costs.
- Some requirements seem reasonable first.
- The problem statement is just a starting point for understanding problem.
- Subsequent analysis helps in fully understanding the problem and its implication.

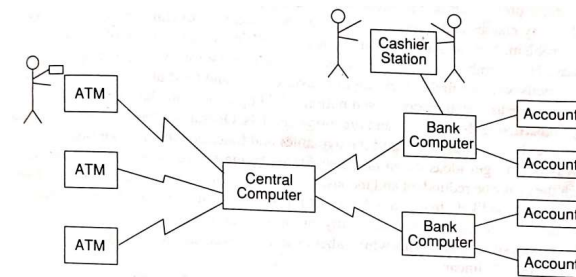


Figure 8.3 ATM network

ATM System: Problem Statement

- Design the software to support a computerized banking network including human cashiers and automatic teller machines (ATMs) to be shared by a consortium of banks. Each bank provides its own computer to maintain its own accounts and process transactions against them. Cashier stations are owned by individual banks and communicate directly with their own bank's computers. Human cashiers enter account and transaction data.
- Automatic teller machines communicate with a central computer that clears transactions with the appropriate banks. An automatic teller machine accepts a cash card, interacts with the user, communicates with the central system to carry out the transaction, dispenses cash, and prints receipts. The system requires appropriate recordkeeping and security provisions. The system must handle concurrent accesses to the same account correctly.
- The banks will provide their own software for their own computers; you are to design the software for the ATMs and the network. The cost of the shared system will be apportioned to the banks according to the number of customers with cash cards.

Object Modeling

- The first step in analyzing the requirements is to construct an object model.
- It describes real world object classes and their relationships to each other.
- Information for the object model comes from the problem statement, expert knowledge of the application domain, and general knowledge of the real world.
- If the designer is not a domain expert, the information must be obtained from the application expert and checked against the model repeatedly.

•The following steps are performed in constructing an object model:

- Identify objects and classes
- Prepare a data dictionary
- Identify associations between objects (including aggregations)
- Identify attributes of object links
- Organize and simplify object classes using inheritance
- Verify that access paths exist for likely queries
- Iterate and refine the model
- Group classes into modules

1. Identifying Object Classes

- The first step in constructing an object model is to **identify relevant object classes** from the application domain.
- The objects include **physical entities** as well as **concepts**.
- All classes must **make sense** in the application domain; avoid computer implementation constructs such as linked lists.
- **Not all** classes are **explicit** in the problem statement; some are **implicit** in the application domain or general knowledge.

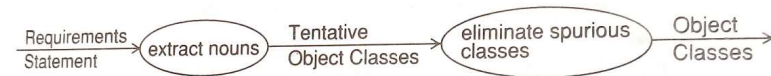


Figure 8.4 Identifying object classes

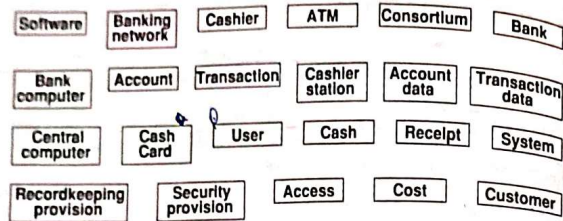


Figure 8.5 ATM classes extracted from problem statement nouns

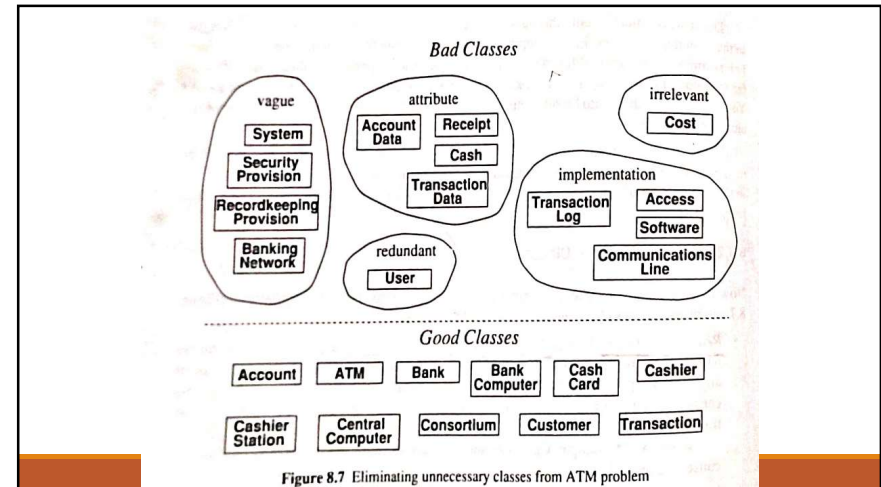


Figure 8.6 ATM classes identified from knowledge of problem domain

2. Keeping the Right classes

- Discard the unnecessary and incorrect classes according to the following criteria:
- Redundant** Classes - If two classes express the same information, the most descriptive name should be kept. (**Customer**, User)
- Irrelevant** Classes - If a class has little or nothing to do with the problem, it should be eliminated. (cost)
- Vague** Classes - A class should be specific. Some tentative classes may have ill defined boundaries or to be too broad in scope (Recordkeeping (vague) part of transaction)

- **Attributes** - Names that primarily describe individual objects should be restated as attributes. If independent existence of a property is important, then make it a class and not an attribute. (account data, cash, receipt)
- **Operations** - If a name describes an operation that is applied to objects and not manipulated in its own right, then it is not a class.
- **Roles** - The name of a class should reflect its intrinsic nature and not a role that it plays in an association. (Person/Customer instead of owner)
- **Implementation Constructs** - Constructs extraneous to real world should be eliminated from analysis model. They may be needed later during design, but not now. (cpu, subroutine, process, algorithm, interrupt)



•3. Preparing a data dictionary

- Prepare a data dictionary for all modeling entities .
- Write a paragraph precisely describing each object class.
- Describe the **scope** of the class within the current problem, including any assumptions or restrictions on its membership or use.
- Data dictionary also describes **associations**, **attributes** and **operations**.

Account—a single account in a bank against which transactions can be applied. Accounts may be of various types, at least checking or savings. A customer can hold more than one account.

ATM—a station that allows customers to enter their own transactions using cash cards as identification. The ATM interacts with the customer to gather transaction information, sends the transaction information to the central computer for validation and processing, and dispenses cash to the user. We assume that an ATM need not operate independently of the network.

Bank—a financial institution that holds accounts for customers and that issues cash cards authorizing access to accounts over the ATM network.

Bank computer—the computer owned by a bank that interfaces with the ATM network and the bank's own cashier stations. A bank may actually have its own internal network of computers to process accounts, but we are only concerned with the one that talks to the network.

Cash card—a card assigned to a bank customer that authorizes access of accounts using an ATM machine. Each card contains a bank code and a card number, most likely coded in accordance with national standards on credit cards and cash cards. The bank code uniquely identifies the bank within the consortium. The card number determines the accounts that the card can access. A card does not necessarily access all of a customer's accounts. Each cash card is owned by a single customer, but multiple copies of it may exist, so the possibility of simultaneous use of the same card from different machines must be considered.

from different machines must be entered.

Cashier—an employee of a bank who is authorized to enter transactions into cashier stations and accept and dispense cash and checks to customers. Transactions, cash, and checks handled by each cashier must be logged and properly accounted for.

Cashier station—a station on which cashiers enter transactions for customers. Cashiers dispense and accept cash and checks; the station prints receipts. The cashier station communicates with the bank computer to validate and process the transactions.

Central computer—a computer operated by the consortium which dispatches transactions between the ATMs and the bank computers. The central computer validates bank codes but does not process transactions directly.

Consortium—an organization of banks that commissions and operates the ATM network. The network only handles transactions for banks in the consortium.

Customer—the holder of one or more accounts in a bank. A customer can consist of one or more persons or corporations; the correspondence is not relevant to this problem. The same person holding an account at a different bank is considered a different customer.

Transaction—a single integral request for operations on the accounts of a single customer. We only specified that ATMs must dispense cash, but we should not preclude the possibility of printing checks or accepting cash or checks. We may also want to provide the flexibility to operate on accounts of different customers, although it is not required yet. The different operations must balance properly.

4. Identifying Associations

- Next step is to identify associations between classes .
- Any **dependency** between **two or more classes** is an association.
- A **reference** from **one class to another** is an association.
- Associations often correspond to **stative verbs or verb phrases**.
- These include **physical location** (next to, part of), **directed actions** (drives), **communication** (talks to), **ownership** (has, part of) or **satisfaction of some kind** (works for).
- **Extract** all the candidates from the problem statement and don't try to refine things too early.
- Don't spend much time trying to distinguish between association and aggregation.
- Majority of the associations are taken from **verb phrases** in the **problem statement**.
- **Associations** which **depend** on **real world knowledge** or **assumptions** must be **verified** with the requestor, as they are not in the problem statement.

Verb phrases:

Banking network includes cashiers and ATMs
 Consortium shares ATMs
 Bank provides bank computer
 Bank computer maintains accounts
 Bank computer processes transaction against account
 Bank owns cashier station
 Cashier station communicates with bank computer
 Cashier enters transaction for account
 ATMs communicate with central computer about transaction
 Central computer clears transaction with bank
 ATM accepts cash card
 ATM interacts with user
 ATM dispenses cash
 ATM prints receipts
 System handles concurrent access
 Banks provide software
 Cost apportioned to banks

Implicit verb phrases:

Consortium consists of banks
 Bank holds account
 Consortium owns central computer
 System provides recordkeeping
 System provides security
 Customers have cash cards

Knowledge of problem domain:

Cash card accesses accounts
 Bank employs cashiers

Figure 8.9 Associations from ATM problem statement

Dynamic Modeling:

- It shows the **time-dependent behavior of the system and the objects** in it.
- Begin dynamic analysis by looking for **events**—externally visible **stimuli** and **responses**.
- Summarize permissible event sequences for each object with a **state diagram**.
- Dynamic model is **insignificant** for a purely **static data repository**.
- It is **important** in the case of **interactive systems**.
- For most problems, **logical correctness** depends on the **sequences of interactions**, not the exact time of interactions.
- **Real time systems** do have specific **timing requirements** on interactions that must be considered during analysis.

Steps performed in constructing a dynamic model:

- Prepare scenario of typical interaction sequences
- Identify events between objects
- Prepare an event trace for each scenario
- Build a state diagram
- Match events between objects to verify consistency

(i) Preparing a Scenario

- Prepare one or more typical **dialogs** between user and system to get a feel for **expected system behavior**.
- These scenarios show the **major interactions, external display format and information exchanges**.
- Approach the dynamic model by scenarios, to ensure that important steps are not overlooked and that the overall flow of the interaction is smooth and correct.
- Sometimes the problem statement describes the full interaction sequence, but most of the time you will have to invent the interaction format.

- First **prepare scenarios for “normal “** cases , interactions without any unusual inputs or error conditions
- Then consider **“special “cases**, such as omitted input sequences, maximum and minimum values, and repeated values.
- Then consider **user error cases**, including invalid values and failures to respond.
- For many interactive applications, error handling is the most different part of the implementation.

- If possible, **allow** the user to **abort an operation or roll back** to a well defined starting point at each step.
- Finally consider various other kinds of interactions that can be overlaid on basic interactions, such as help requests and status queries.
- A scenario is a sequence of events .An event occurs when information is exchanged between an object in the system and an outside agent, such as user. The values exchanged are parameters of the event. The events with no parameters are meaningful and even common. The information in such event is the fact that it has occurred a pure signal. Anytime information is input to the system or output from the system.

- For each **event** identify the **actor** (system, user or other external agent) that caused the event and the parameters of the event.
- The screen layout or output format generally doesn't affect the logic of the interaction or the values exchanged
- Don't worry about output formats for the initial dynamic model; describe output formats during refinement of the model.

The ATM asks the user to insert a card; the user inserts a cash card.
 The ATM accepts the card and reads its serial number.
 The ATM requests the password; the user enters "1234."
 The ATM verifies the serial number and password with the consortium; the consortium checks it with bank "39" and notifies the ATM of acceptance.
 The ATM asks the user to select the kind of transaction (withdrawal, deposit, transfer, query); the user selects withdrawal.
 The ATM asks for the amount of cash; the user enters \$100.
 The ATM verifies that the amount is within predefined policy limits and asks the consortium to process the transaction; the consortium passes the request to the bank, which eventually confirms success and returns the new account balance.
 The ATM dispenses cash and asks the user to take it; the user takes the cash.
 The ATM asks whether the user wants to continue; the user indicates no.
 The ATM prints a receipt, ejects the card, and asks the user to take them; the user takes the receipt and the card.
 The ATM asks a user to insert a card.

Figure 8.15 Normal ATM scenario

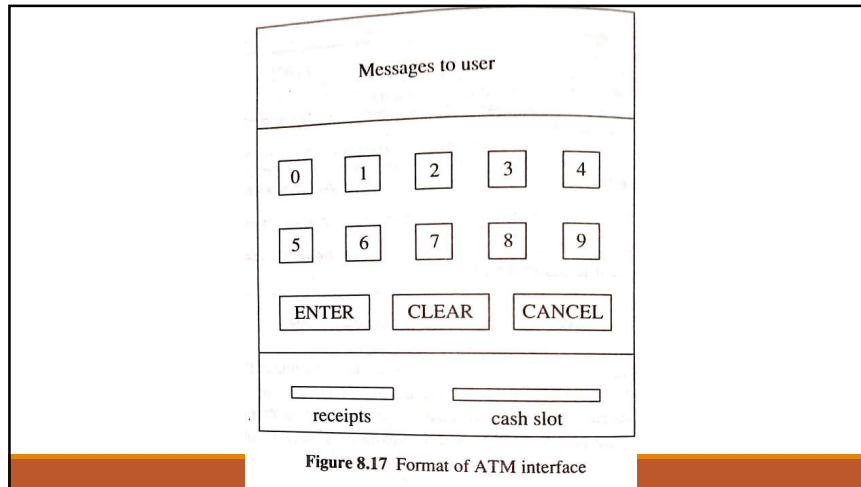
The ATM asks the user to insert a card; the user inserts a cash card.
 The ATM swallows the card and reads its serial number.
 The ATM requests the password; the user enters "3099."
 The ATM verifies the serial number and password with the consortium, which rejects it after consulting the appropriate bank.
 The ATM indicates a bad password and asks the user to reenter it; the user enters "1234" which the ATM successfully verifies with the consortium.
 The ATM asks the user to select the kind of transaction; the user selects withdrawal.
 The ATM asks for the amount of cash; the user has a change of mind and hits "cancel."
 The ATM ejects the card and asks the user to take it; the user takes it.
 The ATM asks a user to insert a card.

Figure 8.16 ATM scenario with exceptions

(ii) Interface Formats

- Most interactions can be separated into two parts:
 - application logic
 - user interface
- Analysis should **concentrate first on the information flow and control**, rather than the presentation format.
- Same program logic can accept input from command lines, files, mouse buttons, touch panels, physical push buttons, or carefully isolated.
- **Dynamic** model captures the **control logic of the application**.

- It is **hard to evaluate** a user interface **without actually testing it**.
- Often the interface can be mocked up so that users can try it.
- **Application logic** can often be **simulated** with **dummy procedures**.
- **Decoupling application logic from user interface** allows **look and feel** of the user interface to be **evaluated** while the application is under development.



(iii) Identifying Events

- Examine the scenario to identify all external events.
- Events include all signals, inputs, decisions, interrupts, transitions, and actions to or from users or external devices.
- Internal computation steps are not events, except for decision points that interact with the external world .
- Use scenarios to find normal events, but don't forget error conditions and unusual events.
- An action by an object that transmits information is an event.
- Most object to object interaction and operations correspond to events.

- Group together under a single name events that have the same effect on flow of control, even if their parameter values differ.
- You may have to construct the state diagram before you can classify all events; some distinctions between events may have no effect on behavior and can be ignored.
- Allocate each type of event to the object classes that send it and receive it.
- The event is an output event for the sender and an input event for the receiver.

- Sometimes an object sends an event to itself, in which case the event is both an output and input for the same class.
- Show each scenario as an event trace – an ordered list of events between different objects assigned to columns in a table.
- If more than one object of the same class participates in the scenario, assign a separate column to each object.
- By scanning a particular column in the trace, you can see the events that directly affect a particular object.
- Only these events can appear in the state diagram for the object.

- Show the **events between a group of classes** (module) on an **event flow diagram**.
- This diagram summarizes events between classes, without regard for sequence.
- Include events from all scenarios, including **error events**.
- The **event flow diagram** is a **dynamic counterpart** to an **object diagram**.
- Paths in the object diagram show possible information flows; paths in the event flow diagram show possible control flows.

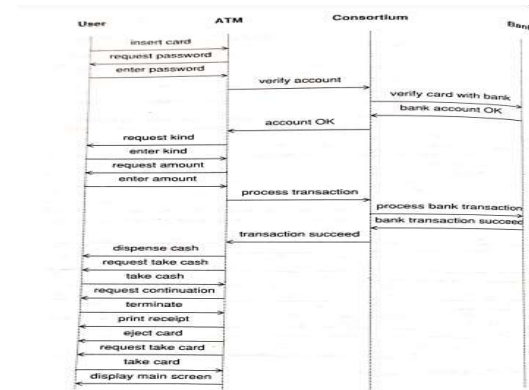


Figure 8.18 Event trace for ATM scenario

(iv) Building a State Diagram

- Prepare a **state diagram** for **each object class** with non trivial dynamic behavior, showing the **events the object receives and sends**.
- Every **scenario or event trace** corresponds to **path through the state diagram**.
- Each **branch in a control flow** is represented by a **state with more than one exit transition**.
- Two paths join at a state if the object “forgets” which the one was taken.

- Beware of two paths that appear identical but which can be distinguished under some circumstances.
- The **judicious use of parameters and conditional transitions** can simplify state diagrams considerably but at the cost of mixing together state information and data.
- State diagrams with too much data dependency can be confusing and counterintuitive.
- Another alternative is to partition a state diagram into two concurrent subdiagrams, using one **subdiagram** for the **main** line and the **other** for the **distinguishing information**.

- After **normal events** have been considered, add **boundary cases** and **special cases**.
- Handling **user errors** needs more thought and code than normal case.
- You are finished with the state diagram of a class when the diagram handles all events that can affect an object of the class in each of its states.
- If there are **complex interactions** with independent inputs, we use **nested** state diagrams.
- Repeat the above process of building state diagrams for **each class of objects**.
- Concentrate on **classes with important interactions**.

- Not all classes need state diagrams.
- Many objects respond to input events independent of their past history, or capture all important history as parameters that do not affect control.
- Such objects may receive and send events.
- List the input events for each object and output event sent in response to each input event, but there will be no further state structure.
- Eventually you may be able to write down state diagram directly without preparing event traces

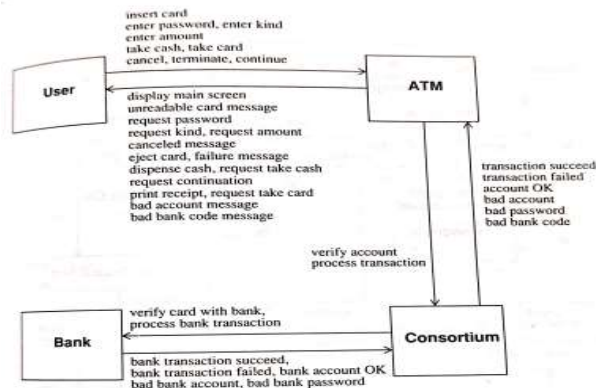


Figure 8.19 Event flow diagram for ATM example

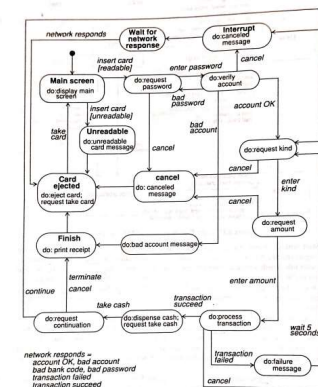


Figure 8.20 State diagram for class ATM

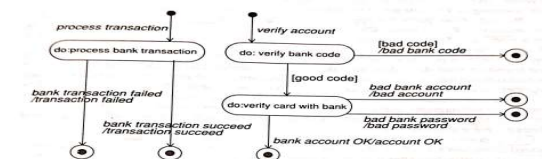


Figure 8.21 State diagram for class Consortium

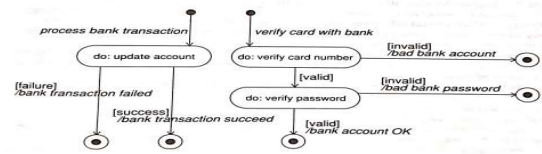


Figure 8.22 State diagram for class Bank

Functional Modeling

- The functional model shows how values are computed, without regard for sequencing, decisions or object structure.
- It shows which values depend on which other values and which function relate them.
- The DFDs are useful for showing functional dependencies.
- Functions are expressed in various ways, including natural language, mathematical equations and pseudo code.
- Processes on DFD corresponds to activities or actions in the state diagrams of the class.
- Flows on DFD correspond to objects or attribute values in object diagram.
- Construct the functional model after the two models.

Steps performed in constructing functional model:

- Identifying input and output values
- Building DFDs showing functional dependencies
- Describe functions
- Identify constraints
- Specify optimization criteria

(i) Identifying input and output values

- List input and output values. (parameters of events between system and outside world.)
- Find any input or output values missed.

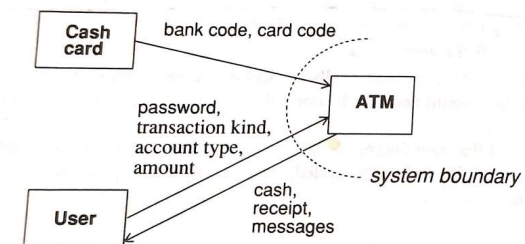


Figure 8.23 Input and output values for ATM system

(ii) Building DFD

- Now construct a DFD, showing how each output value is computed from input values.
- A DFD is usually **constructed in layers**.
- The **top layer may consist of a single process**, or perhaps one process to gather inputs, compute values, and generate outputs.
- Within each DFD layer, work backward from each output value to determine the function that computes it.

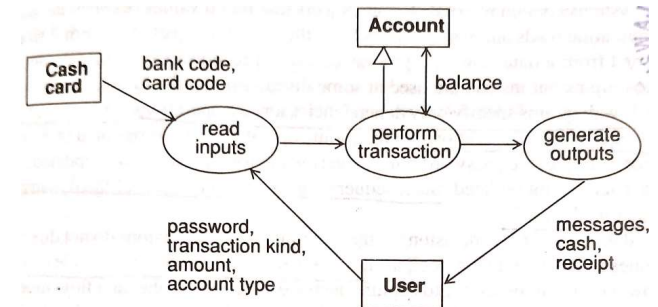


Figure 8.24 Top level data flow diagram for ATM

- If the inputs to the operations are all inputs of the entire diagram, you are done; otherwise some of the operation inputs are intermediate values that must be traced backward in turn.
- You can also trace forward from inputs to outputs, but it is usually harder to identify all users of an input, than to identify all the sources of an output

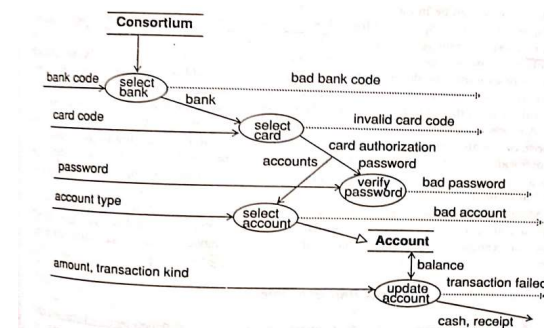


Figure 8.25 Data flow diagram for ATM perform transaction process

- **Expand** each non trivial process in the **top** level DFD into a **lower** level DFD.
- Most systems contain internal storage objects that retain values between iterations.
- An internal store can be distinguished from a data flow or a process because it receives values that don't result in immediate outputs but are used at some distant time in the future.
- The DFD show **only dependencies among operations**. They don't show decisions.(control flows)

iii) Describing Functions

- When the DFD has been refined enough, write a **description** of each **function**.
- The description can be in **natural language, mathematical equations, pseudocode, decision tables**, or some appropriate form.
 - Focus on **what the function** does, and not how it is implemented.
 - The description can be **declarative or procedural**.
 - A declarative description specifies the relationships between input and output values and the relationships among output values.

update account (account, amount, transaction-kind) -> cash, receipt, message

If the amount on a withdrawal exceeds the current account balance,
reject the transaction and dispense no cash

If the amount on a withdrawal does not exceed the current account balance,
debit the account and dispense the amount requested

If the transaction is a deposit
credit the account and dispense no cash

If the transaction is a status request
dispense no cash

In any case,
the receipt shows ATM number, date, time, account number,
transaction-kind, amount transacted (if any), and new balance

Figure 8.26 Function description for update account function

System Design

- System design is the **high-level strategy** for **solving the problem** and **building a solution**. The steps involved in system design are:
 - 1. Estimating system performance**
 - Prepare a rough performance estimate to achieve high accuracy, but simply to determine if the system is feasible.
 - 2. Making a reuse plan**
 - There are 2 different aspects of reuse –
 1. using existing things and
 2. creating reusable new things.
 - It is much easier to reuse existing things than to design new things.
 - Reusable things include models, libraries, patterns, frameworks etc.

The system designer must make the following decisions:

- Organize the system in to subsystems.
- Identify concurrency inherent in the problem.
- Allocate subsystem to processors and tasks.
- Choose an approach for management of data store.
- Handle access to global resources.
- Choose the implementation of control in software.
- Handle boundary conditions.
- Set trade-off priorities.

Breaking a system into subsystems

- First step of system design is to **divide** the system **in a small number of subsystems**.
- Each **subsystem** encompasses aspects of the system that **share some common property** such as similar functionality, same physical location, execution on same kind of hardware.
- A subsystem is a **package of classes, associations, operations, events and constraints interrelated** and have a well defined small interface with other subsystems.
- **Subsystems** are **identified** by the **services it provides**.
- Determining subsystems allow a **division of labor** so each can be designed by a separate team

➤ Most **interactions** should be **within subsystems** rather than across boundaries and it **reduces dependencies** among subsystems.

➤ Relationship between two subsystems can be **client-supplier** or **peer-to-peer**.

- a. In a **client-supplier** relationship, the client calls on the supplier for service. Client must know the interface of the supplier but not vice versa.
 - b. In a **peer-to-peer** relationship, each can call on the other and therefore they must know each others interfaces.
 - c. One way communication (client-supplier) is much easier to build, understand and change than two way (Peer-to-Peer).
- The decomposition of systems into subsystems may be organized as a sequence of layers or partitions.

1.LAYERS

- A layered system is an ordered set of virtual worlds, each built in terms of the ones below it and providing the basis of implementation for the ones above it.
- The objects in each layer can be independent , although there is often some correspondence between objects in different layers.
- Knowledge is one way only: A subsystem knows about the layers below it , but has no knowledge of the layers above it.
- Layered architecture come in two forms: closed and open

- In closed architecture, each layer is built only in terms of the immediate lower layer. This reduces the dependencies between the layers and allows changes to be made most easily because a layer's interface only affect the next layer.
- In an open architecture, a layer can use features of any lower layer to any depth. This reduces the need to redefine operations at each level, which can result in a more efficient and compact code.
- Usually only the top and bottom layers are specified by the problem statement: the top is the desired system, the bottom is the available resources.

2. PARTITIONS

- Partitions vertically divide a system in to several independent or weakly coupled subsystems, each providing one kind of service.
- For example, a computer operating system includes a file system, process control, virtual memory_management, and device control.
- The subsystems may have some knowledge of each other, but this knowledge is not deep, so major dependencies are not created.
- A system can be successively decomposed into subsystems using both layers and partitions in various possible combinations: Layers can be partitioned and partition can be layered.

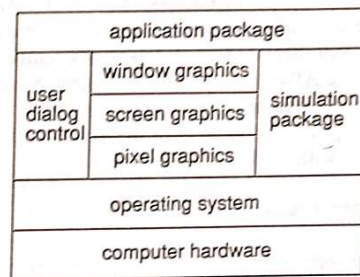


Figure 9.1 Block diagram of a typical application

3. System Topology

- When the top level subsystems are identified, the designer should show the information flow among subsystems with a data flow diagram. Sometimes, all subsystems interact with all other subsystems, but often the flow is simpler.
- For example, many computations have the form of pipeline; a compiler is an example. Other systems are arranged as a star, in which a master subsystem controls all interactions with other subsystems. Use simple topologies when they are suitable for a problem to reduce the number of interactions among subsystems.

Identifying Concurrency

- Two objects are said to be **inherently concurrent** if they can **receive events at the same time without interacting**.
- If the events are unsynchronized, the objects cannot be folded into a single thread of control. (engine and wing control on an airplane must operate concurrently)
- Two subsystems that are **inherently concurrent** need **not** necessarily be implemented as **separate hardware units**.
- The purpose of **interrupts, Operating System and tasking mechanism** is to simulate logical concurrency in a uniprocessor - so if there are no timing constraints on the response then a multitasking Operating System can handle the computation.

14

- All objects are conceptually concurrent, but many objects are interdependent.
- By examining the **state diagram** of individual objects and the **exchange of events** between them, **many objects** can be folded into a **single thread of control**.
- A thread of control is a path through a set of state diagrams on which only a **single object at a time is active** - thread remains in a state diagram until an object sends an event to another object and waits for another event.
- On each thread of control only a single object is active at a time.
- Threads of control are implemented as tasks in computer systems.

15

Allocating Subsystems To Processors & Tasks

- Each **concurrent** system must be **allocated** to a **h/w unit**-general purpose processor or a specialized functional unit.

1. Estimating Hardware Resource Requirements

- The decision to use **multiple processors or hardware functional units** is based on the need for **higher performance** than a single CPU can provide (or fault tolerant requests).
- **Numbers of processors** determine the **volume of computations** and **speed** of the machine.
- The system designer must estimate the **required CPU power** by computing the **steady state load** as the **product of the number of transactions per second and the time required to process a transaction**.
- **Experimentation** is often useful to assist in estimating.

16

2. Hardware/Software Tradeoffs

- **Hardware** can be viewed as a **rigid** but highly **optimized** form of **software**.
- The **system designer** must decide which **subsystems** will be implemented in **hardware** and which in **software**.
- **Subsystems** are in **hardware** for 2 reasons:
 - Existing hardware provides exactly the functionality required – e.g. floating point processor
 - Higher performance is required than a general purpose CPU can provide.
- **Compatibility, cost, performance, flexibility** are considerations whether to use hardware or software.
- **Software** in many cases may be **more costly than hardware** because the cost of people is very high.
- **Hardware** is much **more difficult to change** than software.

17

3. Allocating Tasks To Processors

Tasks are assigned to processors because:

- Certain tasks are required at specific physical locations
- Response time or information flow rate exceeds the available communication bandwidth between a task and a piece of hardware.
- Computation rates are too great for single processor, so tasks must be spread among several processors.

4. Determining Physical Connectivity

- Choose the topology of connecting the physical units.
- Choose the topology of repeated units.
- Choose the form of connection channels & the communication protocols.

Management Of Data Stores

- The internal and external data stores in a system provide clean separation points between subsystems with well defined interfaces.
- Data stores may combine data structures, files, and database implemented in memory or on secondary storage devices.
- The different data stores provide trade-offs between cost, access time, capacity and reliability.
- Files are cheap, simple and permanent form of data store.
- File operations are low level and applications must include additional code to provide a suitable level of abstraction.
- File implementations vary from machine to machine so portability is more difficult.

- Databases are powerful and make applications easier to port.
- Databases do have a complex interface and may work awkwardly with programming languages.
- The kind of data that belongs in a Database:
 - Data that requires access at fine levels of detail by multiple users
 - Data that can be efficiently managed by DBMS commands
 - Data that must be ported across many hardware and operating system platforms
 - Data that must be accessible by more than one application program.

- The kind of **data** that belongs in a **file** & **not** in a **relational DB**:
 - Data that is **voluminous** but **difficult** to **structure** within the confines of a Database.
 - Data that is **voluminous** in quantity and of **low information density** (debugging data, historical files)
 - **Raw data** that is summarized in the Database
 - **Volatile data** that is kept a short time and then discarded.

10

1. Advantages of using a Database

- Handles crash recovery, concurrency, integrity
- Common interface for all applications
- Provides a standard access language - SQL

2. Disadvantages of using a Database

- Performance overhead
- Insufficient functionality for advanced applications - most databases are structured for business applications.
- Awkward interface with programming languages.

11

Handling Global Resources

- The system designer must **identify the global resources** (processors, tape drives, etc) and determine **mechanisms** for **controlling access to them**.
- If the resource is a **physical object**, it can control itself by establishing a **protocol** for obtaining access within a concurrent system.
- If the resource is **logical entity** (object ID) there is a danger of **conflicting access in a shared environment**.
- Independent tasks could simultaneously use the same object ID.
- Each **global resource** must be **owned** by a **guardian object** that controls access to it.
- A **logical resource** can also be **partitioned** logically, such that **subsets** are **assigned to different guardian objects** for independent control.

12

Common Architectural Frameworks

- There are several prototypical architectural frameworks that are common in existing systems.
- Each of these is well-suited to a certain kind of system.
- If we have an application with similar characteristics, we can save effort by using the corresponding architecture, or at least use it as a starting point for our design.

13

The kinds of system include:

1. Batch transformation – a data transformation executed once on an entire input set.
2. Continuous transformation – a data transformation performed continuously as inputs change.
3. Interactive interface – a system dominated by external interactions.
4. Dynamic simulation – a system that simulates real world objects.
5. Real time system – a system dominated by strict timing constraints
6. Transaction manager – a system concerned with storing and updating data, often including concurrent access from different physical locations

•The kinds of systems include:

1. Batch transformation:

- It is a data transformation executed once on an entire input set.
- A batch transformation is a sequential input-to-output transformation, in which inputs are supplied at the start, and the goal is to compute an answer.
- There is no ongoing interaction with the outside world.
- The most important aspect of the batch transformation is the functional model which specifies how input values are transformed into output values.

The steps in designing a batch transformation are:

- Break the overall transformation into stages, each stage performing one part of the transformation.
- The system diagram is a **data flow diagram**. This can be taken from the functional model.
- Define intermediate object classes for the data flows between each pair of successive stages.
- Expand each stage in turn until the operations are straightforward to implement.
- Restructure the final pipeline for optimization.

2. Continuous Transformation:

- It is a system in which the outputs actively depend on changing inputs and must be periodically updated.
- Because of time constraints, the entire set of outputs cannot usually be recomputed each time an input changes.
- the new output values must be computed incrementally. The transformation can be implemented as a pipeline of functions.
- The effect of each incremental change in an input value must be propagated through the pipeline.

The steps in designing a pipeline for continuous transformation are:

- Draw a DFD for the system.
 - The input and output actors correspond to data structures whose value change continuously.
 - Data stores within the pipeline show parameters that affect the input-to-output mapping.
- Define intermediate objects between each pair of successive stages.
- Differentiate each operation to obtain incremental changes to each stage.
- Add additional intermediate objects for optimization.

3. Interactive Interface:

- An interactive interface is a system that is dominated by interactions between the system and external agents, such as humans, devices or other programs.
- The external agents are independent of the system, so their inputs cannot be controlled.
- Interactive interfaces are dominated by the dynamic model.

The steps in designing an interactive interface are:

- Isolate the objects that form the interface from the objects that define the semantics of the application.
 - Use predefined objects to interact with external agents, if possible.
 - Use the dynamic model as the structure of the program.
 - Isolate physical events from logical events.
 - Fully specify the application functions that are invoked by the interface.
- Make sure that the information to implement them is present.

4. Dynamic Simulation:

- A dynamic simulation models or tracks objects in the real world.
- The steps in designing a dynamic simulation are:
 - Identify actors, real-world objects from the object model-the actors have attributes that are periodically updated.
 - Identify discrete events. Discrete events correspond to discrete interactions with the object and can be implemented as operations on the object.
 - Identify continuous dependencies.
 - Generally a simulation is driven by a timing loop at a fine time scale. Discrete events between objects can often be exchanged as part of the timing loop.

5.Real Time System:

- Real Time System is an interactive system for which time constraints on actions are particularly tight or in which the slightest timing failure cannot be tolerated.

21

6.Transaction Manager:

- A transaction manager is a database system whose main function is to store and access information.
- The information comes from the application domain.
- Most transaction managers must deal with multiple users and concurrency.
- The steps in designing a transaction management system are:
 - Map the object model directly into a database.
 - Determine the units of concurrency. Introduce new classes as needed.
 - Determine the unit of transaction.
 - Design concurrency control for transactions.

22